

Fraud Detection Project — Q&A Log

Planning log for the internship ML + software project: an explainable, real-time fraud detection platform (PaySim dataset, autoencoder + GNN + SHAP)

1. Give me a project topic for ML + software roles

Suggested fintech/analytics-relevant ideas: Credit Risk / Loan Default Predictor, Stock/Crypto Trend Predictor with sentiment, Fraud Detection System (anomaly detection + dashboard), Resume Screener, Expense Tracker with spend prediction. Recommended Fraud Detection / Credit Risk as strongest resume fit for fintech ML Engineer roles.

2. Give a bigger, industry-oriented, current, even crazy-difficult project

Proposed: AI-powered Fraud Detection & Risk Monitoring Platform using Graph Neural Networks (GNN) to detect fraud rings, on top of a baseline anomaly detector. Components: data layer (transaction graph), ML core (Isolation Forest / Autoencoder baseline + GNN for fraud rings), explainability (SHAP), real-time simulation (WebSocket), full-stack dashboard (graph visualization), and an optional MLOps monitoring touch. GNN treated as a stretch goal if time runs short.

3. Full info

Provided: problem statement, dataset choices (Kaggle PaySim or IEEE-CIS), tech stack per layer (Pandas/NetworkX for data, Scikit-learn + PyTorch for ML, PyTorch Geometric for GNN, SHAP for explainability, Node/Express + Flask backend, Socket.io real-time, React + D3.js dashboard, Render.com deployment), a 15-day day-wise plan, evaluation metrics (precision/recall/F1/ROC-AUC, not accuracy, due to class imbalance), and a draft resume bullet.

4. Add a 'wow factor' — make it stand out, not generic

Suggested differentiators: (1) adversarial angle — a fraud-pattern generator that adapts against the model, (2) natural-language SHAP explanations instead of raw values, (3) animated temporal replay showing a fraud ring growing over time, (4) tiered risk alerts (low/medium/high with different actions) instead of a binary flag, (5) a 'model vs human' interactive benchmark. Recommended combo: GNN + natural-language explanations + tiered alerts for strongest impact with reasonable added complexity.

5. Detailed problem statement — what real problem it solves and what the system does

Real-world problem: banks process huge transaction volumes, fraud patterns evolve, black-box flags aren't regulator-friendly, and most systems miss coordinated fraud rings by only looking at single transactions. System response: models transactions as a graph, flags individual anomalies (autoencoder) and connected fraud rings (GNN), explains every flag (SHAP + plain language), and

serves results via a real-time dashboard with tiered actions (log / review / auto-hold).

6. Formal problem statement + solution (for resume/SIH)

Delivered a formal Problem Statement paragraph, a Proposed Solution paragraph, and a one-line pitch: 'A real-time, explainable fraud detection system that catches not just suspicious transactions, but suspicious networks.' — ready to paste into a resume or SIH submission.

7. Dataset

Recommended Kaggle PaySim (Synthetic Financial Datasets for Fraud Detection): ~6.3 million transactions over 744 steps (30-day simulation), isFraud making up only ~0.13% of rows. Explained key columns (step, type, amount, nameOrig, nameDest, balances, isFraud) and how nameOrig/nameDest map directly to graph nodes. Suggested IEEE-CIS as a heavier alternative if time allows, but PaySim as the practical 15-day choice.

8. The dataset / live data should be from one organization

Suggested framing PaySim as the internal data of a single fictional fintech (e.g. 'NovaBank'): rename/wrap account IDs with a bank prefix, replay transactions in time order as that bank's live feed, and state clearly in the README that the system is scoped to monitor one institution's internal transaction graph (matching how such systems are actually deployed in practice).

9. Didn't understand — who commits fraud, who catches it, where does data come from, what does the system do

Explained in plain terms: fraudsters are either external account-takeover attackers or organized mule-account networks (money laundering rings); the bank's fraud/risk analyst team is who catches it, using an ML system (this project) instead of manual review. Since real bank data isn't accessible for a student project, the system replays the historical PaySim CSV row-by-row (ordered by the step/time column) to simulate a live feed, scores each transaction, flags rings, and shows everything on a dashboard — compared to a security guard replaying CCTV footage and raising an alarm on suspicious activity.

10. What kind of fraud does it detect

Listed concrete fraud types covered: account takeover / unauthorized cash-out, the classic PaySim TRANSFER→CASH_OUT pattern, money-muling / fraud rings (via GNN), structuring / amount-splitting, and sudden behavior-change (velocity fraud). Also flagged honestly what it does NOT catch: phishing/social-engineering fraud and identity-theft at account-opening (KYC) stage, since transaction data alone can't reveal those.

11. So what's the tech stack for all this

Consolidated full stack: Pandas + NetworkX (data/graph), Scikit-learn Isolation Forest + PyTorch/Keras Autoencoder (anomaly detection), PyTorch Geometric / GraphSAGE or GCN (fraud ring detection), SHAP (explainability), Node.js + Express plus a Python Flask/FastAPI microservice (backend), Socket.io (real-time streaming), React + D3.js/vis-network + Chart.js (dashboard), MongoDB/PostgreSQL (storage), Render.com (deployment). Gave a priority order for the 15-day timeline (must-haves vs stretch goals like the GNN).

12. Where to get the dataset and how to test it

Dataset: Kaggle page [kaggle.com/datasets/ealaxi/paysim1](https://www.kaggle.com/datasets/ealaxi/paysim1), download via website or Kaggle API; suggested sampling a subset (e.g. 200k–500k stratified rows) for fast development before using the full file. Testing: chronological train/test split, evaluate with precision/recall/F1/ROC-AUC/confusion matrix (not raw accuracy, given the ~0.13% fraud rate), test the replay script and dashboard end-to-end, check edge cases, and pre-select a few clear demo examples for a smooth live presentation.

13. After training, where does the 'live stream' data actually come from

Clarified that there is no real live bank feed; the held-out test set (data the model never saw during training) is replayed row-by-row in time order with small delays via a WebSocket script, simulating a live feed. Recommended stating this transparently ('a replay layer standing in for what would be a Kafka-style real-time event stream in production') as a system-design strength rather than a weakness.

14. Shared the PaySim Kaggle dataset page ('this is one')

Confirmed the shared file matches the PaySim dataset described earlier: ~6.36M rows, 8,213 fraud cases (0.13%), CASH_OUT (35%) and PAYMENT (34%) as the dominant transaction types, and sample rows showing the classic TRANSFER→CASH_OUT fraud pattern. Suggested next steps: download, load with Pandas, filter `isFraud==1` to confirm the pattern, then begin building the `nameOrig`→`nameDest` graph.

15. Shared the CSV header row — explain what each column stores

Walked through every column: `step` (time in hours), `type` (5 transaction categories), `amount`, `nameOrig`/`nameDest` (graph source/destination nodes), `oldbalanceOrg`/`newbalanceOrig`/`oldbalanceDest`/`newbalanceDest` (pre/post balances), `isFraud` (the actual model target), and `isFlaggedFraud` (PaySim's own simple rule-based flag on transfers over \$200,000, useful only as a weak baseline for comparison, not as a model input). Warned about data leakage risk from the balance columns, since they tend to zero out on fraud transactions, and suggested engineering safer features like `balance_diff` instead of using them raw.
